

Introduction to Python

- Good if you want to write small programs quickly and be able to manage the complexity of larger ones
- Is like the shell, only better
- Freely available for many platforms
- Much slower than C/C++ or Fortran
- But it's relatively easy to call C/C++ and Fortran libraries from Python
- Comes with many standard libraries
- Python is an object-oriented language, from the ground up

- Python is a simple interpreted language
- Python runs interactively from a command shell by typing `python`
- You can put commands in a file and have Python execute the file
- Python is relaxed about the type of data assigned to variables
- However variables must have values before they are used

Python Object Types

Object type	Example literals/creation
Numbers	1234, 3.1415, 3+4j, Decimal, Fraction
Strings	'spam', "guido's", b'a\x01c'
Lists	[1, [2, 'three'], 4]
Dictionaries	{'food': 'spam', 'taste': 'yum'}
Tuples	(1, 'spam', 4, 'U')
Files	myfile = open('eggs', 'r')
Sets	set('abc'), {'a', 'b', 'c'}
Other core types	Booleans, types, None

- The Python interpreter command-line syntax is:

```
python {options} [-c command | -m module | file | -] {arguments}
```

where the file contains python scripts, run directly, and modules, imported.

- When you run python without a script argument, python enters an interactive session and prompts you to enter Python statements or expressions.
- The prompt string is `>>>` when Python expects a statement or expression and `...` when a statement or expression has been started but not completed.

Running Python Codes: Example

```
shahriar@shahriar-ThinkPad:~/Documents/Teaching/HPC/Python/Python$ python
Python 2.7.15rc1 (default, Apr 15 2018, 21:51:34)
[GCC 7.3.0] on linux2
Type "help", "copyright", "credits" or "license" for more information.
>>> print('Hello world!')
Hello world!
>>> print(2*8)
16
>>> lumber = 'jack'
>>> lumber
'jack'
>>> 2*8
16
>>> quit()
shahriar@shahriar-ThinkPad:~/Documents/Teaching/HPC/Python/Python$ python
Python 2.7.15rc1 (default, Apr 15 2018, 21:51:34)
[GCC 7.3.0] on linux2
Type "help", "copyright", "credits" or "license" for more information.
>>> import os #load the precoded module os
>>> get.cwd()
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
NameError: name 'get' is not defined
>>> os.getcwd()
'/home/shahriar/Documents/Teaching/HPC/Python/Python'
>>> for x in 'spam' #The interactive prompt runs one statement at a time
      File "<stdin>", line 1
        for x in 'spam' #The interactive prompt runs one statement at a time
                                ^
SyntaxError: invalid syntax
>>> for x in 'spam': #The interactive prompt runs one statement at a time
...     print x
...
s
p
a
m
>>> for x in 'spam': #The interactive prompt runs one statement at a time
...     print x
      File "<stdin>", line 2
        print x
            ^
IndentationError: expected an indented block
>>> import sys
>>> print(sys.platform)
linux2
>>> █
```

```

shahriar@earth: ~/Teaching/Courses/GraduateSummer/Python
File Edit View Search Terminal Tabs Help
shahriar@earth: ~/Teaching/Courses/GraduateSummer/Python
shahriar@earth:~/Teaching/Courses/GraduateSummer/Python$ python
Python 2.7.3 (default, Apr 10 2013, 05:46:21)
[GCC 4.6.3] on linux2
Type "help", "copyright", "credits" or "license" for more information.
>>> 1+2
3
>>> print 1+2
3
>>> print 'Shahriar ' + 'Afkhami'
Shahriar Afkhami
>>> x='Instructor: '
>>> y='Shahriar Afkhami'
>>> z=x+y
>>> print z
Instructor: Shahriar Afkhami
>>>
shahriar@earth:~/Teaching/Courses/GraduateSummer/Python$ nano print.py
shahriar@earth:~/Teaching/Courses/GraduateSummer/Python$ python print.py
Instructor: Shahriar Afkhami
3
twotwotwo
5
Traceback (most recent call last):
  File "print.py", line 22, in <module>
    print 2 + str(3)
TypeError: unsupported operand type(s) for +: 'int' and 'str'
shahriar@earth:~/Teaching/Courses/GraduateSummer/Python$ python print.py
Instructor: Shahriar Afkhami
3
twotwotwo
5
Traceback (most recent call last):
  File "print.py", line 22, in <module>
    print 2 + str(3)
TypeError: unsupported operand type(s) for +: 'int' and 'str'
shahriar@earth:~/Teaching/Courses/GraduateSummer/Python$ nano print.py
shahriar@earth:~/Teaching/Courses/GraduateSummer/Python$ python print.py
Instructor: Shahriar Afkhami
3
twotwotwo
5
23

```

```
shahriar@earth: ~/Teaching/Courses/GraduateSummer/Python
File Edit View Search Terminal Tabs Help
shahriar@earth: ~/Teaching/Courses/GraduateSummer/Python
GNU nano 2.2.6 File: print.py

x = 'Instructor: '      # assign 'Instructor: ' string to x
y = 'Shahriar Afkhami'  # assign 'Shahriar Afkhami' to y
z = x + y               # concatenate strings
print z
z = 3
print z
print 'two'*z
# z + 'two' is not allowed
z = int('2') + 3
print z
print 2 + str(3)
print 2**2              # Exponentiation
print 13%5              # Remainder

z = 100
z += 1
z *= 5

^G Get Help      ^O WriteOut      ^R Read File      ^Y Prev Page      ^K Cut Text
^X Exit          ^J Justify       ^W Where Is       ^N Next Page      ^U UnCut Text
```

Program Execution

- Python is an interpreted language, as opposed to a compiled one, though the Python interpreter automatically compiles Python source files as needed.
- When you execute a python code, Python first compiles your source code (internally and hidden from you).
- The format is known as bytecode.
- Python does not save the compiled bytecode form of a script when you run the script directly; rather, Python recompiles the script each time you run it.
- This bytecode translation is performed to speed execution.
- Python saves the compiled bytecode file with the same basename and extension .pyc.
- The saved bytecode optimizes startup speed. The next time you run your program, Python will load the .pyc files and skip the compilation step, as long as you haven't changed your source code since the byte code was last saved.

Strings are used to record textual information as well as arbitrary collections of bytes. They are our first example of what we call a sequence in Python that is, a positionally ordered collection of other objects.

```
>>> S = 'Spam'
>>> len(S)           # Length
4
>>> S[0]             # The first item in S, indexing by zero-based position
'S'
>>> S[1]             # The second item from the left
'p'
```

strings are immutable in Python they cannot be changed in-place after they are created.

String Methods

capitalize	Capitalize first letter of string
lower/upper	Convert all letters to lower/uppercase
count	Count how many times one string appears in another
find	Return index of first occurrence of one string in another
replace	Replace occurrences of one string with another

The Python list object is the most general sequence provided by the language. Lists are positionally ordered collections of arbitrarily typed objects, and they have no fixed size. They are also mutable.

append	Add to the end of the list.
count	Count how many times something appears in the list.
index	Find the first occurrence of something in the list.
insert	Insert something into the list.
remove	Remove the first occurrence of something from the list.
reverse	Reverse the list in place.
sort	Sort the list in place.

<code>x.append(99)</code>	<code>['c', 'a', 'b', 'c', 99]</code>
<code>x.count('c')</code>	<code>2</code>
<code>x.index('a')</code>	<code>1</code>
<code>x.insert(2, 'diamond')</code>	<code>['c', 'a', 'diamond', 'b', 'c']</code>
<code>x.remove('c')</code>	<code>['a', 'b', 'c']</code>
<code>x.reverse()</code>	<code>['c', 'b', 'a', 'c']</code>
<code>x.sort()</code>	<code>['a', 'b', 'c', 'c']</code>

Python dictionaries are not sequences, but are instead known as mappings. Mappings are also collections of other objects, but they store objects by key instead of by relative position.

Example: consider the following three-item dictionary (with keys food, quantity, and color)

```
D = {'food': 'Spam', 'quantity': 4, 'color': 'pink'}
```

```
D['quantity'] += 1 # Add 1 to 'quantity' value
```

```
D = {'food': 'Spam', 'color': 'pink', 'quantity': 5}
```

Dictionaries can be changed in-place freely.

The tuple objects are like a list that but they cannot be changed, like strings.

```
T = (1, 2, 3, 4)
```

```
len(T)
```

```
4
```

```
T + (5, 6)
```

```
(1, 2, 3, 4, 5, 6)
```

```
T[0]
```

```
1
```

```
T[0] = 2 # ERROR: Tuples are immutable
```

Modules

- Create a one-line Python module file called `hello.py`, containing:

```
title = "Hello World"
```

- Then start python and run:

```
import hello
```

```
>>> print(hello.title)
```

```
Hello World
```

- A module is mostly just a package of variable names, functions, classes, variables, and so on, that are intended to be used in other files and other programs.
- The names within that package are called attributes – an attribute is simply a variable name that is attached to a specific object (like a module).

```
>>> print(hello.title)
```

- In general, the dot expression syntax:

`object.attribute`

lets you fetch any attribute attached to any object.

- Alternatively:

```
>>> from hello import title
```

```
>>> print(title)
```

- This becomes more useful when you start defining objects such as functions and classes in your modules: such objects become reusable software components that can be accessed by name from one or more client modules.

From Wiki

Object-oriented programming (OOP) is a programming paradigm based on the concept of “objects”, which may contain data, in the form of fields, often known as attributes; and code, in the form of procedures, often known as methods. A feature of objects is that an object’s procedures can access and often modify the data fields of the object with which they are associated. In OOP, computer programs are designed by making them out of objects that interact with one another. There is significant diversity of OOP languages, but the most popular ones are class-based, meaning that objects are instances of classes, which typically also determine their type.

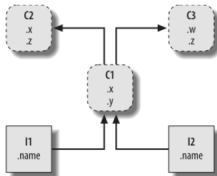
Classes and Object-Oriented Programming

- Classes: the definitions for the data format and available procedures for a given type or class of object; may also contain data and procedures (known as class methods) themselves, i.e. classes contain the data members and member functions
- Objects: instances of classes
- Encapsulation: binds together the data and functions that manipulate the data, and that keeps both safe from outside interference and misuse.
- Inheritance: properties from the general category of all classes. These common properties need to be implemented only once for the general case and can be reused by all types of objects we may build in the future.
- Composition: objects can contain other objects in their instance variables.

Classes and Object-Oriented Programming

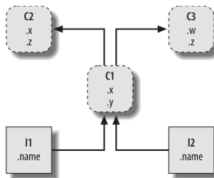
- In Python, classes are created with a new statement: the class statement.
- most of the OOP story in Python boils down to this expression:

`object.attribute`



- What do `I1.x` and `I2.x`, `I1.y` and `I2.y` return?
- What do `I1.z` and `I2.z`, `I1.name` and `I2.name` return?

Class Method Calls



- If `I2.w` reference is a function call, then it means: call the `C3.w` function to process `I2`; i.e. call `I2.w()` into the call `C3.w(I2)`, passing in the instance as the first argument to the inherited function.

Coding Class Trees

```
class C2: ...           # Make class objects (ovals)
class C3: ...
class C1(C2, C3): ...   # Linked to superclasses

I1 = C1()              # Make instance objects (rectangles)
I2 = C1()              # Linked to their classes
```

- Each class statement generates a new class object.
- Each time a class is called, it generates a new instance object.
- Instances are automatically linked to the classes from which they are created.
- Classes are linked to their superclasses by listing them in parentheses in a class header line; the left-to-right order there gives the order in the tree.

Coding Class Trees

```
class C1(C2, C3):  
    def setname(self, who):  
        self.name = who  
  
I1 = C1()  
I2 = C1()  
I1.setname('bob')  
I2.setname('mel')  
print(I1.name)
```

Make and link class C1
Assign name: C1.setname
Self is either I1 or I2

Make two instances

Sets I1.name to 'bob'
Sets I2.name to 'mel'
Prints 'bob'

- Attributes are usually attached to classes by assignments made within class statements, and not nested inside function definition (def) statements.
- Attributes are usually attached to instances by assignments to a special argument passed to functions inside classes, called `self`.

Coding Class Trees

```
class C1(C2, C3):           # Make and link class C1
    def setname(self, who): # Assign name: C1.setname
        self.name = who    # Self is either I1 or I2

I1 = C1()                  # Make two instances
I2 = C1()
I1.setname('bob')          # Sets I1.name to 'bob'
I2.setname('mel')          # Sets I2.name to 'mel'
print(I1.name)             # Prints 'bob'
```

The first argument in a class method function is special because it always receives the instance object that is the implied subject of the method call. Its usually called `self` by convention. Because method functions always have this implied subject object context by default, we say they are object-oriented, i.e., designed to process or change objects.

Coding Class Trees

```
class C1(C2, C3):                # Make and link class C1
    def setname(self, who):      # Assign name: C1.setname
        self.name = who         # Self is either I1 or I2

I1 = C1()                        # Make two instances
I2 = C1()

I1.setname('bob')                # Sets I1.name to 'bob'
I2.setname('mel')                # Sets I2.name to 'mel'
print(I1.name)                   # Prints 'bob'
```

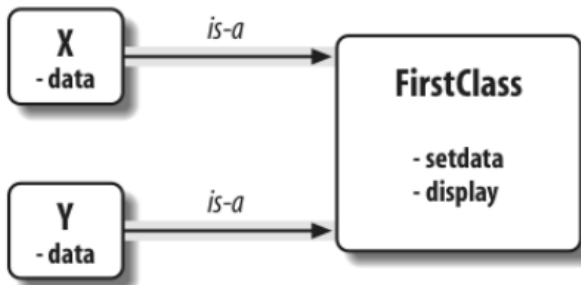
Operationally, when a `def` appears inside a class like this, it is usually known as a method, and it automatically receives a special first argument, called `self` by convention, that provides a handle back to the instance to be processed.

Classes and Object-Oriented Programming

```
>>> class FirstClass:           # Define a class object
...     def setdata(self, value): # Define class methods
...         self.data = value    # self is the instance
...     def display(self):
...         print(self.data)     # self.data: per instance
...
>>> x = FirstClass()           # Make two instances
>>> y = FirstClass()           # Each is a new namespace
```

Functions inside a class are usually called methods. They're coded with normal `def`.

Classes and Object-Oriented Programming



```
>>> x.setdata("King Arthur")  
>>> y.setdata(3.14159)
```

```
# Call methods: self is x  
# Runs: FirstClass.setdata(y, 3.14159)
```

- What does `x.display()` return?
- What does `y.display()` return?
- What does `x.data = "New value"` do?

Inheritance

```
>>> class SecondClass(FirstClass):           # Inherits setdata
...     def display(self):                   # Changes display
...         print('Current value = "%s"' % self.data)
...
>>> z = SecondClass()
>>> z.setdata(42)                           # Finds setdata in FirstClass
>>> z.display()                             # Finds overridden method in SecondClass
Current value = "42"
```

The net effect here is that `SecondClass` specializes `FirstClass` by changing the behavior of the `display` method. On the other hand, `SecondClass` (and any instances created from it) still inherits the `setdata` method in `FirstClass`.

We call this act of replacing attributes by redefining them lower in the tree overloading.

Inheritance

The specialization introduced in SecondClass is completely external to FirstClass. That is, it doesn't affect existing or future FirstClass objects, like the `x` from the prior example:

```
>>> class SecondClass(FirstClass):           # Inherits setdata
...     def display(self):                   # Changes display
...         print('Current value = "%s"' % self.data)
...

>>> z = SecondClass()
>>> z.setdata(42)                           # Finds setdata in FirstClass
>>> z.display()                             # Finds overridden method in SecondClass
Current value = "42"

>>> x.display()                             # x is still a FirstClass instance (old message)
New value
```